

Code Inspection Report v0.1

Discovering Early Hollywood

Samuel Backer



Inspectors: PaperTrail: Arius Ahmad, Robert Kulow, Vasu Patel, Anthony Veilleux, Ben Yandell

Coders: Willy N' Gang: Xander Dufour, Liam Hillery, Vincent Lin, Patrick Storer, Caleb Thurston

Table of Contents

1. Introduction.....	3
1.1 Purpose of This Document.....	3
References.....	3
1.2 Coding and Commenting Conventions - Ben.....	3
1.3 Defect Checklist.....	4
2. Code Inspection Process.....	5
2.1 Description.....	5
2.2 Impressions of the Process.....	6
2.3 Inspection Meetings.....	6
3. Modules Inspected - Anthony.....	7
4. Defects.....	9
Recommended Technical Fixes.....	11
Appendix A – Team Review Sign-off.....	12
Appendix B – Document Contributions.....	14

1. Introduction

1.1 Purpose of This Document

The purpose of this Code Inspection Report is to document the findings and observations from a formal inspection of the Discovering Early Hollywood software. The inspection was conducted by the PaperTrail team to evaluate the quality, security, and maintainability of the code developed by Willy N' Gang. The report identifies potential defects, consistency in coding and style, and gives recommendations for improvement as we move closer to final release. The audience for this document is the development team, the project stakeholder, the instructor, and the future maintainers of this software. This document summarizes the recorded inspection process, module reviews, coding conventions, and a checklist of potential defects observed during the inspection.

References

Software Requirements Specification (SRS) - Discovering Early Hollywood
System Design Document SDD - Discovering Early Hollywood
Python PEP 8 Style Guidelines
PostgreSQL Documentation

1.2 Coding and Commenting Conventions

The inspected software uses a stack of Python, JavaScript, HTML, PostgreSQL. The backend is implemented using Python with Flask framework. '[app.py](#)' acts as the main entry point for the application and is used for routing logic. The frontend is built using HTML with JavaScript providing the interactions for client-side functionality. Data is handled using a PostgreSQL database. Testing of the web interface is performed using Selenium to simulate user interactions and verify frontend functionality. Modules `db_utils.py` and `db_auth.py` are supporting modules to contain database access and authentication logic, separating responsibilities between the application routing and the data management.

The development team appears to follow consistent coding practices across the inspected modules. The Python code follows regular conventions with lowercase variable naming and structured functions. Style looks consistent across the different modules, with no real emphasis on deficiencies during the inspection.

An important factor related to this team's project is the inheritance of legacy code from a previous capstone development team. Due to this, the team had to work with the existing code and rework what was possible to aid them in the development.

During the inspection, it was noted that there was minimal error handling. The primary reporting came from the database level. Logging mechanisms outside of PostgreSQL appeared limited. It would benefit not only the user but also the development team if error handling were made more robust.

1.3 Defect Checklist

Category	Possible Defect
Logic Errors	Database schema defined; not populated
Logic Errors	Unhandled edgecases in authentication logic
Logic Errors	Incomplete feature implementation (Flags)
Logic Errors	Non-working links during navigation
Security	SQL Injection vulnerabilities in database queries
Security	Improper privilege management or authorization checks
Security	Weak password validation/ error messaging
Error Handling	Generic exception handling without logging
Error Handling	Errors not logged outside of Database
Error Handling	Non-specific authentication error messages
Maintainability	Dependency on legacy framework from previous development team
Testing	Limited validation of front end inputs
Testing	Insufficient testing for backend logic
Coding Conventions	Lack of standardized docstrings for functions
Coding Conventions	Inconsistent formatting across modules
User Experience	Missing user feedback for fail or successful actions

Rebuttal

Team Willy N' Gang will be rebutting the code inspection report made by Team PaperTrail by using `"*blue_text"`, For Example

`*The sky is blue on a clear day*`

We appreciate the time and effort put into reviewing our codebase and acknowledge that some of the critiques raised valid points that's worth addressing. However, we also believe that some claims are potentially wrong or based on an incomplete reading of our code. Therefore in the sections of blue we shall address the critique and provide context where necessary to support our positions.

2. Code Inspection Process

The code inspection process outlines the state of the code, how the inspection went, and lastly, where and when code inspections occurred. By the end of this section, readers should understand how effective the code inspection was in determining the verification and validation of the project. Did the team build the product right, and were they building the right product? Where best coding practices followed? And what were some of the design decisions they made to get around pitfalls? These are the goals intended to be answered in the code inspection.

2.1 Description

Before the formal code inspection, the inspecting team brainstormed some questions. One such question was why there were so many hard-coded values present in their scripts that didn't seem to change through the script or by user interaction. Another was why have large blocks of string variables containing data plainly presentable in their HTML, why not just grab the data from a database to reduce overhead for HTTP? As not every user is going to have great internet access, making the HTML as small as possible will assist in load times. These were some of the opening questions the inspection team wanted to start with, but incidentally, the actual team had made substantial changes to their GitHub before the code inspection, resulting in most of the inspection team's initial questions being disregarded. As a result, they had the reader go through each line of code again to prompt the inspection team for questions; this may be the explanation as to why the newer question may seem a little on the basic side. But this "non-ideal" disruption allowed the inspection team to have a much more intimate experience with the coding team, as questions could be formed and asked quickly to get immediate answers for the authors. This caused an arguably more effective approach as everyone had a friendly attitude and wasn't

pressured to have a concrete answer. Thus, it led to the code inspection report to fill in the logical gaps and questions/answers that were left out during the formal code inspection.

Some of the newer questions presented asked why the development team decided to use Python as their backend language. As they are building a website, JavaScript, HTML, and CSS are the commonly used languages. The inspection team was interested in why they diverged from this standard. Their response was due to the fact that they inherited this project from a previous team, and it was they who decided to use Python this way. Following, the question posed next was, what were the challenges of using Python instead of JavaScript? Turns out there are several problems with using Python this way, as most web browsers don't natively support Python in HTTP and won't run automatically. The development team circumvented this by hosting a server that is called by the HTML to get its Python scripts. The initial question of why they had large blocks of string variables containing data plainly presentable in their HTML was resolved, as the development team had pivoted to using a SQL database to easily store and make quick queries to retrieve documents. The main roadblock they were running into with the database was getting legal copyright information from their client, so they were permitted to store and share said information. Their database is responsible for housing all film documentation from 1912 to 1928, meaning they needed a quick and easy database. So when the inspection team then questioned why they decided to use SQL, their answer was simply that SQL databases are excellent at storing and retrieving large amounts of flat data.

2.2 Impressions of the Process

While the code inspection was meant to be conducted in a formal fashion, the unexpected changes to the code beforehand led the review to take a more informal approach. Each member quickly realized that we are all still learning this process, and some slack was to be given. This caused the review to be much more palpable for everyone, and jokes could be passed to alleviate the previous tensions. Which is arguably better for this is a learning experience for students in this capstone course. The inspection team was able to ask more down-to-brass-tacks questions about the design decisions. Overall, creating a better experience for each member during the review. Some improvements that could be implemented for future students is to freeze their code at a deadline before the code inspection, so this wouldn't be likely to occur again. Also, better communication on who is undertaking each role; what each role is meant to do would cause a smoother flow for the code inspection.

The coding team highlighted that app.py was the main script that allowed their software to work, as such the best and worst modules arise from said file. Firstly, the “best” unit would be their valid_id function on line 59, as its function is to take in the variable doc_id, which is a string and perform a “fullmatch” operation and return a bool if it was successful, and it contains an edge-case check to see if the data is null beforehand. This means it will almost never throw errors and should be flawless. Then the “worst” unit would be the index function on line 101, as just the sheer size of the function is substantial. It performs several tasks and calls various functions from other Python scripts, and ends with a return that calls another function, render_template, and takes in 13 parameters. On top of all that, there is a search query to the database that might return the wrong values or timeout if their database isn't set up to be normalized. It doesn't take much to see how this function could go wrong at several points, leading it to be the highlight that could cause major flaws in the program.

2.3 Inspection Meetings

The formal code inspection was conducted on March 4th, 2026, in room 214 of the Ferland Engineering Education and Design Center at the University of Maine, Orono, on monitor 14. The inspection elapsed from 2:05pm - 2:35pm, allowing for 30 minutes of inspection time. Participants were the Willy N' Gang team, consisting of Xander Dufour, Liam Hillery, Vincent Lin, Patrick Storer, and Caleb Thurston. As well as the PaperTrail team, consisting of Arius Ahmad, Robert Kulow, Vasu Patel, Anthony Veilleux, and Ben Yandell. Authors were Liam Hillery and Caleb Thurston, Moderator was Xander Dufour, Reader was Vincent Lin, Recorder was Patrick Storer and Ben Yandell, lastly, the Inspectors were the PaperTrail team, Arius Ahmad, Robert Kulow, Vasu Patel, and Anthony Veilleux. The inspection went over their db_utils.py, db_auth.py, and app.py Python scripts, as well as their index.html, base.html, view_history.html, and documents_manager.html HTML files. They claimed that these were the most important coding scripts that allowed their software to function.

* This team listed our team's supposed assigned roles, but we weren't the ones inspecting the code. The roles were meant to be for the inspecting team, not the team being inspected, and their roles are not listed properly. *

3. Modules Inspected

This section describes all of the modules from team Willy N' Gang that were inspected by team Papertrail during the code inspection. Each module outlines its purpose and scale for the software.

app.py

This is the main entry point for the application. It initializes the Flask web server, defines the HTTP routes that the user interface interacts with, and coordinates the logic by calling helper functions.

This module implements the Web Server component described in Section 2.1 (Architectural Design) and Section 4 (Requirements Matrix). It acts as the "Service Layer," receiving requests from the UI (Frontend) and routing them to the appropriate logic.

App.py accurately implements the planned upload PDF feature.

One difference that was noticed when comparing the Decomposition Description and App.py was that UserManager and DocumentManagers utility is included in App.py

datatypes.py

A collection of classes and types for storing document, query, and user data.

This directly maps the classes that were defined in section 2.2 of the SDD (Decomposition Description)

Currently the 'User' class is commented out and is stated that the DB will handle it.

** We will remove the 'User' class following this report, this file has seen little modification recently and, resultingly, we didn't notice the unused class. **

db_utils.py

Contains helper functions for interacting with the PostgreSQL database, particularly for generating dynamic SQL queries based on search filters.

Implements the logic ascribed to the DocumentManager and DatabaseManager in the SDD (Section 2.2).

db_auth.py

A collection of helpers for user authentication against the PostgreSQL database

Implements the UserManager logic and "User Account Database" requirements (Section 4)

uploadData.py

CLI script for batch uploading documents, metadata, and transcripts from the local filesystem to the database.

Fulfills FR 8 (Add Entry) and acts as the "Database Manager" tool for populating the system.

tableDefinitions.py

Defines the database schema definition.

Implements Section 3.1 (Persistent Data Design)

Overall matches Section 3.1 very well.

4. Defects

The main purpose of this section is to highlight some possible defects that the inspection team found during and after the code inspection. The coding team should review and rectify any issues stated below for the integrity of the software.

Name of Module	Defect Description	Defect Area
db_utils.py	SQL Injection Risk (Dynamic Table Names): In relation_from_id_to_all_values, the relation and idColumn strings are formatted directly into SQL. Since these come from the Query object, a malicious actor could inject table names to leak sensitive data.	Correctness / Security

app.py	Hardcoded Default Secret Key: The line <code>app.secret_key = os.environ["FLASK_SECRET"]</code> or <code>os.urandom(20)</code> is safe, but the environment check often fails in local dev, and the app lacks a strict "Production" check, potentially leading to session hijacking if a static key is ever used.	Other (Security)
tableDefinitions.sql	Weak Password Storage: The users table defines <code>encoded_password</code> as <code>char(20)</code> . Modern secure hashes (like those generated by <code>werkzeug.security</code>) are much longer than 20 characters. This will cause password truncation and login failure.	Correctness
db_auth.py	Plaintext Comparison Potential: In <code>get_user_password_hash</code> , the code returns the hash directly. If the coder later compares this using <code>==</code> instead of <code>check_password_hash</code> , it creates a timing attack vulnerability.	Other (Security)
app.py	Resource Leak (DB Connection): The <code>dbConnection</code> is opened at the global scope but never explicitly closed using an <code>at exit</code> handler or Flask <code>teardown</code> context. This can lead to connection "hanging" on the database server.	Other (Refactoring)

db_utils.py	PEP 257 Violation: Several helper functions (e.g., get_document) lack the mandatory summary line in their docstrings, or use inconsistent indentation within the docstring itself.	Commenting Convention
datatypes.py	Inconsistent Naming (PEP 8): The Metadata and Flag classes use camelCase for attributes (e.g., reporterName, errorLocation). Python convention strictly requires snake_case for instance variables.	Coding Convention
app.py	Silent Error Masking: In the /save_ocr route, the except Exception as e block returns a 500 error with the string of the error but does not log the stack trace. This makes debugging "Permission Denied" or "File Not Found" errors nearly impossible.	User Friendliness
db_utils.py	Logic Error (Lowercase doc_id): In get_document, the code calls doc_id.lower() on every query. If the LoC (Library of Congress) IDs in the database are case-sensitive or stored in Uppercase, this function will consistently return None.	Correctness

uploadData.py	Brittle Error Handling: The main function catches a generic Exception to check if tables exist. This would catch <i>any</i> error (like a wrong password), print "tables already exist!", and proceed to fail silently during the data load.	Correctness / Other
---------------	--	---------------------

*

Defect # 1 SQL sanitization: We are using psycopg2, so all calls to psycopg2.SQL.Literal() or psycopg2.SQL.Identifier() (which are used for “relation”, “idColumn”, “valueColumn”, etc.) are restricted to known safe values before functions are ever called.

Defect # 2 Hardcoded Default Secret Key: This is for early development and will be changed in production.

Defect # 3 Weak Password Storage: This has been changed in our own database and forgot to change in the tableDefinitions.sql, and are very thankful to be warned about it.

Defect # 4 Plaintext Comparison Potential: If we did use “==” as comparison instead of check_password_hash() then it will create a timing attack vulnerability, but we use check_password_hash so we should be fine.

Defect # 5 Resource Leak (DB Connection): Very good catch, we fixed this defect in the time between the initial code inspection and this report, in a separate branch.

Defect # 6 PEP 257 Violation: Good catch, we have a lot of functionality in the db_utils.py that have doc strings and we could have missed some.

Defect # 7 Inconsistent Naming (PEP 8) : We will include this change in our cleanup of datatypes.py.

Defect # 8 Silent Error Masking: This is a snippet of code that was carried over from the previous capstone project. In our project it is out of scope, and has since been removed.

Defect # 9 Logic Error: The `doc_id.lower()` is used because we have control over the naming convention, so we have changed it to be all lowercase.

Defect # 10 Brittle Error Handling: The content of this function is largely temporary since it was written before we had access to the final data format for this project (which we gained on the day of the inspection.) When it is rewritten to allow the upload of the current files, we will take this error handling into account and ensure that the correct error triggers this behavior.

*

Appendix A – Team Review Sign-off

All team members have reviewed this Code Inspection Report (CIR) and agree with all its content. The team acknowledges that the document accurately reflects the agreed-upon project scope and requirements at this development stage. Any minor issues shall be recorded among the team in the comment section.

This is a living document which is subject to changes or modifications. In the event that any addendums to the document, which redefine the scope or requirements of this project, will have to require every team member to agree and sign off.

Signatures:	Date:	Signature:
Robert Kulow	Mar. 11, 2026	 Robert Kulow (Mar 30, 2026 13:07:33 EDT)
Vasu Patel	Mar. 11, 2026	 Vasupatel (Mar 30, 2026 13:16:51 EDT)
Ben Yandell	Mar. 11, 2026	 Benjamin Yandell (Mar 30, 2026 13:59:45 EDT)
Arius Ahmad	Mar. 11, 2026	 Arius Ahmad (Mar 30, 2026 13:45:48 EDT)
Anthony Veilleux	Mar. 11, 2026	 Anthony Veilleux (Mar 30, 2026 13:10:00 EDT)

Appendix B – Document Contributions

This section outlines the individual contributions made by each team member to the development of the document. It provides a record of who was responsible for each specific section for accountability throughout the documentation process. The following shows what they contributed and the percentage of work each member did.

Xander Dufour:

- Contributions: rebuttal
- Percentage: 4%

Liam Hillery:

- Contributions: rebuttal
- Percentage: 4%

Vincent Lin:

- Contributions: rebuttal
- Percentage: 4%

Patrick Storer:

- Contributions: rebuttal
- Percentage: 4%

Caleb Thurston:

- Contributions: rebuttal
- Percentage: 4 %

Arius Ahmad

- Contributions: Appendices and Editing/Formatting
- Percentage: 16%

Robert Kulow

- Contributions: Section 2
- Percentage: 16%

Vasu Patel

- Contributions: Section 4
- Percentage: 16%

Anthony Veilleux

- Contributions: Section 3
- Percentage: 16%

Ben Yandell

- Contributions: Section 1
- Percentage: 16%